Applied Data Mining

— Week 9—

Spring 2026

Dr. Hashim Safdar

Outline

- Machine Learning Algorithms
  - K-Nearest Neighbor
- Confusion Matrix
- Evaluation Metrics
- Cross-validation, hold-out method

K- Nearest Neighbor

K- Nearest Neighbor (K-NN)

- K-NN is an algorithm that can be used when we have a bunch of objects that have already been classified.

- There are other similar objects that haven't been classified yet, and we want a way to automatically label them.

- Whether linear regression would work? for linear regression, the output must be a continuous variable.

- In contrast, here the output of your algorithm is categorical label.

K- Nearest Neighbor (K-NN)

- The intuition behind k-NN is to consider the *most similar* other objects defined in terms of their attributes, look at their labels, and give the unassigned object the majority vote.

- If there's a tie, you can randomly select among the labels.

K- Nearest Neighbor (K-NN)

- For example, if you had a bunch of movies that were labeled as “thumbs up” or “thumbs down,” and you have a movie called “Data Gone Wild” that hadn’t been rated yet.

You could look at its attributes:

- “Length of movie, genre, number of horror scenes, number of Oscar winning actors in it, and budget”.

- You can then find other movies with similar attributes, look at their ratings, and then give “Data Gone Wild” a rating without ever having to watch it.

K-NN Process

- To automate this process, two decisions must be made:

- First, how do you define similarity or closeness?

    - Once you define it for a given unrated object, take the most similar objects and call them neighbors, who each have a “vote.”

- Second decision: how many neighbors should you consider? This value is k.

Similarity or Distance Metrics

- Definitions of “closeness” and similarity vary and depend on the context:

- “Closeness in social networks could be defined as the number of overlapping friends”,

- For example, for our problem of what a neighbor is, we can use Euclidean distance if the variables are on the same scale.

Similarity or Distance Metrics

- **Euclidean Distance:** Simplest, for attributes that are real-valued, fast to calculate.

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}$$

- **Cosine Similarity:** Can be used between two real-valued vectors, *x* and *y* , and will yield a value between –1 (exact opposite) and 1 (exactly the same) with 0 in between meaning independent. Good for documents, images, etc.

$$d(x, y) = 1 - \frac{\vec{x} \cdot \vec{y}}{||x|| \, ||y||}$$

Similarity or Distance Metrics

- **Euclidean Distance:** Simplest, for attributes that are real-valued, fast to calculate.

- **Cosine Similarity:** Can be used between two real-valued vectors, *x* and *y* , and will yield a value between –1 (exact opposite) and 1 (exactly the same) with 0 in between meaning independent. Good for documents, images, etc.

$$d(x, y) = 1 - \frac{\vec{x} \cdot \vec{y}}{||x|| \, ||y||}$$

Similarity or Distance Metrics

- **Euclidean Distance:** Simplest, for attributes that are real-valued, fast to calculate.

- **Cosine Similarity:** Can be used between two real-valued vectors, *x* and *y* , and will yield a value between –1 (exact opposite) and 1 (exactly the same) with 0 in between meaning independent. Good for documents, images, etc.

$$d(\vec{x}, \vec{y}) = \sqrt{(\vec{x} - \vec{y})^T S^{-1} (\vec{x} - \vec{y})},$$

Example 1 (K-NN)

We have following student data:

S#	Math	CS	Result
1	4	3	Fail
2	6	7	Pass
3	7	8	Pass
4	5	5	Fail
5	8	8	Pass

Query: X = (Math = 6, CS = 8, Result = ?)

Example 1 (K-NN)-Solution

- **Euclidean Distance:** Simplest, for attributes that are real-valued, fast to calculate.

$$d = \sqrt{|x_{o_1} - x_{A1}|^2 + |x_{o_2} - x_{A2}|^2}$$

- **Cosine Similarity:** Can be used between two real-valued vectors, *x* and *y* , and will yield a value between –1 (exact opposite) and 1 (exactly the same) with 0 in between meaning independent. Good for documents, images, etc.

$$d(x,y) = 1 - \frac{\vec{x} \cdot \vec{y}}{||x||\,||y||}$$

Example 1 (K-NN)-Solution

I $\sqrt{(6-4)^2+(8-3)^2} = \sqrt{29} = 5.38$

II $\sqrt{(6-6)^2+(8-7)^2} = 1$

III $\sqrt{(6-7)^2+(8-8)^2} = 1$

IV $\sqrt{(6-5)^2+(8-5)^2} = \sqrt{10} = 3.16$

V $\sqrt{(6-8)^2+(8-8)^2} = 2$

Example 1 (K-NN)-Solution

- As K = 3, we select best three neighbors, 2, 3, and 5 as per similarity metric.
- So all three neighbors have class label "Pass" and no neighbor has class label "Fail"
- Therefore, 3 > 0
- So Query: X = (Math = 6, CS = 8, Result = Pass)

Example 2

Apply the K-NN and predict the food type to which Tomato belong?

Ingredient	Sweet	chrunch	Food Type
Grape	8	5	FRUIT
Greenbean	3	7	Vegetable
Nuts	3	6	Protein
Orange	7	3	FRUIT

Query: Tomato = (Sweet = 6, Crunch = 4)

Select k = 1

Example 2 (Solution)

Euclidean distance.

$$D(\text{Tomato, Grape}) = \sqrt{(6-8)^2+(4-5)^2} = \sqrt{5} = 2.2$$

$$D(\text{Tomato, Green bean}) = \sqrt{(6-3)^2+(4-7)^2} = \sqrt{18} = 4.2$$

$$D(\text{Tomato, Nuts}) = \sqrt{(6-3)^2+(4-6)^2} = \sqrt{13} = 3.6$$

$$D(\text{Tomato, Orange}) = \sqrt{(6-7)^2+(4-3)^2} = \sqrt{2} = 1.4$$

Example 2 (Solution)

Since distance of Tomato from Orange is minimum.

∴ Tomato will belong to FRUIT ANS..

K-NN Complete Process

1. Decide about *similarity* or *distance* metric.
2. Split original dataset into training and test data.
3. Pick an evaluation metric. (e.g. accuracy)
4. Run k-NN few times, change *k* and check the evaluation measure.
5. Optimize *k* by picking the one with the best evaluation measure.
6. After choosing *k*, use the same training set and now test set is the record whose label is unknown.

Confusion Matrix, Evaluation Metrics

Model Evaluation

- Evaluation metrics are quantitative measures used to assess the performance and effectiveness of a statistical or machine learning model.
- Use validation/test set of class-labeled instances instead of training set when assessing accuracy
- Methods for splitting the dataset:
  - Holdout method
  - Cross-validation

Evaluating Classifier Accuracy: Holdout & Cross-Validation Methods

Holdout method:

- Divide your data into a training set and a test set: 80% as the training and 20% as the test.
- Fit the model on the training set, then look at the *accuracy* on the test set and compare it to that on the training set.
- If the accuracies are approximately the same, then our model generalizes well and we're not in danger of overfitting.

Evaluating Classifier Accuracy: Holdout & Cross-Validation Methods

- **Cross-validation** (*k*-fold, where k = 10 is most popular)
  - Randomly partition the data into *k mutually exclusive* subsets, each approximately of equal size.
  - At *i*-th iteration, use D_i as the test set and others as the training set

Classifier Evaluation Metrics

- **True positives** (TP): These refer to the positive tuples that were correctly classified by the classifier. Let TP be the number of true positives.

- **True negatives** (TN): These are the negative tuples that were correctly classified by the classifier. Let TN be the number of true negatives.

- **False positives (**FP): These are the negative tuples that were incorrectly classified as positive (e.g., tuples of class buys computer = no for which the classifier predicted buys computer = yes). Let FP be the number of false positives.

Confusion Matrix

- **False negatives** (FN): These are the positive tuples misclassified as negative (e.g., tuples of class buys computer = yes for which the classifier predicted buys computer = no). Let FN be the number of false negatives.

Confusion Matrix:

Actual class\Predicted class	$C_{1\ (Yes)}$	$\neg\ C_{1\ (No)}$
$C_{1\ (Yes)}$	**True Positives (TP)**	**False Negatives (FN)**
$\neg\ C_{1\ (No)}$	**False Positives (FP)**	**True Negatives (TN)**

Confusion Matrix

Example of Confusion Matrix:

Actual class\Predicted class	buy_computer = yes	buy_computer = no	Total
buy_computer = yes	**6954**	**46**	7000
buy_computer = no	**412**	**2588**	3000
Total	7366	2634	10000

- Given *m* classes, an entry, $CM_{i,j}$ in a **confusion matrix** indicates # of tuples in class *i* that were labeled by the classifier as class *j*
- May have extra rows/columns to provide totals

Classifier Evaluation Metrics: Accuracy, Error Rate, Sensitivity and Specificity

A\P	C	¬C	
C	**TP**	**FN**	**P**
¬C	**FP**	**TN**	**N**
	P'	**N'**	**All**

- **Classifier Accuracy,** or recognition rate: percentage of test set tuples that are correctly classified

  Accuracy = (TP + TN)/All

- **Error rate:** *1 – accuracy*, or

  Error rate = (FP + FN)/All

- Class Imbalance Problem:
  - One class may be *rare*, e.g. fraud, or HIV-positive
  - Significant *majority of the negative class* and minority of the positive class

  Sensitivity: True Positive recognition rate

  - Sensitivity = TP/P

  Specificity: True Negative recognition rate

  - Specificity = TN/N

Classifier Evaluation Metrics: Precision and Recall, and F-measures

- **Precision**: exactness – what % of tuples that the classifier labeled as positive are actually positive

$$precision = \frac{TP}{TP + FP}$$

- **Recall:** completeness – what % of positive tuples did the classifier label as positive?

$$recall = \frac{TP}{TP + FN}$$

- Perfect score is 1.0
- Inverse relationship between precision & recall
- ***F*** **measure (*F_1* or *F*-score)**: harmonic mean of precision and recall,

$$F = \frac{2 \times precision \times recall}{precision + recall}$$

Classifier Evaluation Metrics: Example

Actual Class\Predicted class	cancer = yes	cancer = no	Total	Recognition (%)
cancer = yes	**90**	**210**	300	30.00 (*sensitivity*
cancer = no	**140**	**9560**	9700	98.56 (*specificity)*
Total	230	9770	10000	96.50 (*accuracy*)

Precision **= 90/230 *100 = 39.13%,**

Recall **= 90/300 *100 = 30.00%,**

Sensitivity **= 90/300 *100 = 30%,**

Specificity = 9560/9700 *100 = 98.56%

Any

Question